Loading Libraries

```
library(dplyr)
library(caret)
library(randomForest)
library(pROC)
library(ggplot2)
library(ROSE)
library(biomaRt)
library(effsize)
```

Preparing the data

Loading Raw Counts Data and Normalization

The raw RNA-seq count data is loaded into the environment. The dataset consists of the **significant differentially expressed lncRNAs** across **1,224 samples**, including primary tumor (TP) and matched normal tissue (NT) samples. The expression values are currently in raw count format and require normalization prior to downstream analysis. The gene expression matrix was normalized using **z-score scaling**, whereby each gene's expression values were centered and scaled to have a mean of zero and a standard deviation of one.

```
## Reading the samples x genes dataframe
ml_data <- read.csv("lncRNA_DEGs_ml_input.csv", check.names = FALSE)
rownames(ml_data) <- ml_data[, 1]
ml_data <- ml_data[, -1]

head(ml_data[, 1:4])
```

```
##                               type ENSG00000286448 ENSG00000272438
## TCGA.BH.A18H.01A.11R.A12D.07   TP               0               2
## TCGA.E2.A14P.01A.31R.A12D.07   TP               0              17
## TCGA.AN.A04A.01A.21R.A034.07   TP               1               4
## TCGA.AQ.A0Y5.01A.11R.A14M.07   TP               0               2
## TCGA.A8.A07I.01A.11R.A00Z.07   TP               0               6
## TCGA.E9.A1R4.01A.21R.A14D.07   TP               0              17
##                               ENSG00000230699
## TCGA.BH.A18H.01A.11R.A12D.07               7
## TCGA.E2.A14P.01A.31R.A12D.07              27
## TCGA.AN.A04A.01A.21R.A034.07               7
## TCGA.AQ.A0Y5.01A.11R.A14M.07               5
## TCGA.A8.A07I.01A.11R.A00Z.07              24
## TCGA.E9.A1R4.01A.21R.A14D.07              34
```

```
# Normalization requires a genes x samples dataframe
ml_data <- as.data.frame(t(ml_data)) # needs genes x samples dataframe
ml_labels <- as.vector(unlist(ml_data[1, ])) # Storing the labels separately
ml_counts <- ml_data[-1, ] # Counts matrix
ml_counts <- as.data.frame(
  apply(ml_counts, 2, function(x) as.numeric(trimws(x))),
  row.names = rownames(ml_counts)
```

```
) # Converting counts to numeric

## z-score scaling
ml_counts <- scale(ml_counts)

ml_data <- as.data.frame(t(ml_counts))
ml_data$type <- as.factor((ml_labels))
ml_data <- ml_data %>% dplyr::select(type, everything())
```

Stratified Train-Test Split

The normalized gene expression data were partitioned into training and testing sets using a 70:30 split.

```
set.seed(345)
## Creating the index to split
trainIndex <- createDataPartition(
  ml_data$type,
  p = 0.7,
  list = FALSE
)
trainData <- ml_data[trainIndex, ]
testData  <- ml_data[-trainIndex, ]

# Data sample split
table(trainData$type)
```

```
##
##  NT  TP
##  80 778
```

```
table(testData$type)
```

```
##
##  NT  TP
##  33 333
```

Feature Selection

Feature selection was performed using **Cohen's d** on the raw count training data matrix. The histogram provides the range of Cohen's d values for the lncRNA genes. The **top 10 lncRNA genes** with the highest absolute Cohen's d values were selected as discriminative features for the Random Forest model.

```
# Getting a genes x samples matrix
# Extract labels
labels <- trainData$type

# Remove the label column
train_expr <- trainData %>% dplyr::select(-type)
# Transpose - genes as rows, samples as columns
train_expr <- t(train_expr)
```

```r
# Splitting the samples by class
tumor_idx  <- which(labels == "TP")
normal_idx <- which(labels == "NT")

## Compute the cohens d value
cohens_d <- apply(train_expr, 1, function(gene_expr) {
  tumor_vals  <- gene_expr[tumor_idx]
  normal_vals <- gene_expr[normal_idx]

  # Compute Cohen's d
  d <- cohen.d(tumor_vals, normal_vals)$estimate
  return(d)
})

# Obtaining a dataframe
cohens_d_df <- data.frame(
  gene = rownames(train_expr),
  cohens_d = cohens_d
)

## Arranging in descencing order of cohens d value
cohens_d_df <- cohens_d_df %>%
  arrange(desc(abs(cohens_d)))

## Plotting the cohens d histogram
hist(cohens_d_df$cohens_d, xlab = "Cohen's d", main = "Cohen's d Histogram")
```


```
## Subsetting the top 10 highest cohens d lncRNA genes
top_genes_cohens <- cohens_d_df[1:10, ]
rownames(top_genes_cohens) <- NULL
print(top_genes_cohens)
```

```
##                gene  cohens_d
## 1  ENSG00000286289 -4.010616
## 2  ENSG00000241684 -3.840382
## 3  ENSG00000249669 -3.475715
## 4  ENSG00000226005 -3.377158
## 5  ENSG00000280339 -3.262150
## 6  ENSG00000232079 -3.235408
## 7  ENSG00000245812 -3.196890
## 8  ENSG00000235904 -3.102177
## 9  ENSG00000231246 -3.084014
## 10 ENSG00000279204 -2.985642
```

```
top_gene_names <- top_genes_cohens$gene
trainData <- trainData %>%
  dplyr::select(type, all_of(top_gene_names))

## Selecting the test data for the top 10 lncRNA genes only
# Done to ensure same matrix dimensions for train and test
testData <- testData %>%
  dplyr::select(type, all_of(top_gene_names))
```

Data Balancing

Due to class imbalance between primary tumor (TP) and normal tissue (NT) samples, **Synthetic Minority Over-sampling Technique (SMOTE)** was applied to the training set to ensure a more balanced class distribution and reduces model bias during classification.

```
## SMOTE for Data Balancing on training data only
set.seed(199)
trainData_bal <- ROSE(type ~ ., data = trainData, N = 2000, p = 0.5)$data
table(trainData_bal$type)
```

```
##
##   TP   NT
## 1023  977
```

Model Building

The Random Forest classifier was trained on the previously generated training dataset. A **5-fold cross-validation** strategy was implemented and **Hyperparameter tuning** was performed for mtry and ntree to improve model performance, minimize overfitting and improve generalization. Finally, the **top lncRNA diagnostic biomarkers** were identified based on **ranked variable importance scores** derived from the model.

```
## Defining the Training variables
x_rf <- as.matrix(trainData_bal[, -1])
y_rf <- as.factor(trainData_bal[, 1])

## Defining the Testing variables
x_test <- as.matrix(testData[, -1])
y_test <- as.factor(testData[, 1])

y_rf   <- factor(y_rf, levels = c("TP", "NT"))
y_test <- factor(y_test, levels = c("TP", "NT"))

set.seed(345)
## Training Model using 5-fold CV
ctrl <- trainControl(
  method = "cv",
  number = 5,
  classProbs = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

## Performing Hyperparameter Tuning (mtry and ntree)
# mtry
mtry_grid <- expand.grid(
  mtry = seq(2, floor(sqrt(ncol(x_rf))), by = 1)
) # Testing mtry values from 2 up to sqrt(number of features)

# Obtaining best mtry value for our training set
rf_tuned_mtry <- caret::train(
```

```
  x = x_rf,
  y = y_rf,
  method = "rf",
  metric = "ROC",
  trControl = ctrl,
  tuneGrid = mtry_grid,
  ntree = 500,
  importance = TRUE
)

best_mtry <- rf_tuned_mtry$bestTune$mtry
cat("Best mtry =", best_mtry, "\n")
```

```
## Best mtry = 2
```

```
# ntree
# Different values for the number of trees in the Random Forest model
ntree_values <- c(100, 200, 300, 500, 700)
results_ntree <- data.frame()
# Running a loop on the ntree_values to and calculating the ROC for each

for (nt in ntree_values) {

  set.seed(123)

  model <- caret::train(
    x = x_rf,
    y = y_rf,
    method = "rf",
    metric = "ROC",
    trControl = ctrl,
    tuneGrid = data.frame(mtry = best_mtry),
    ntree = nt
  )
  results_ntree <- rbind(results_ntree,
                         data.frame(ntree = nt,
                                    ROC = max(model$results$ROC)))
}

print(results_ntree)
```

```
##   ntree       ROC
## 1   100 0.9995001
## 2   200 0.9994925
## 3   300 0.9995049
## 4   500 0.9994698
## 5   700 0.9994699
```

```
best_ntree <- results_ntree$ntree[which.max(results_ntree$ROC)]
cat("Best ntree =", best_ntree, "\n")
```

```
## Best ntree = 300
```

```r
## Using the above found parameters, tuning our final RF model
set.seed(345)

final_rf <- randomForest(
  x = x_rf,
  y = y_rf,
  mtry = best_mtry,
  ntree = best_ntree,
  importance = TRUE
)

print(final_rf)
```

```
##
## Call:
##  randomForest(x = x_rf, y = y_rf, ntree = best_ntree, mtry = best_mtry,      importance = TRUE)
##                Type of random forest: classification
##                      Number of trees: 300
## No. of variables tried at each split: 2
##
##         OOB estimate of  error rate: 1.05%
## Confusion matrix:
##      TP  NT class.error
## TP 1007  16 0.015640274
## NT    5 972 0.005117707
```

```r
## Predicting on Test Data
pred_class <- predict(final_rf, x_test)

## Evaluation of Model Performance
# Confusion matrix
confusionMatrix(pred_class, y_test)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  TP  NT
##         TP 323   1
##         NT  10  32
##
##                Accuracy : 0.9699
##                  95% CI : (0.9469, 0.9849)
##     No Information Rate : 0.9098
##     P-Value [Acc > NIR] : 4.201e-06
##
##                   Kappa : 0.8369
##
##  Mcnemar's Test P-Value : 0.01586
##
##             Sensitivity : 0.9700
##             Specificity : 0.9697
##          Pos Pred Value : 0.9969
##          Neg Pred Value : 0.7619
```

```
##              Prevalence : 0.9098
##          Detection Rate : 0.8825
##    Detection Prevalence : 0.8852
##       Balanced Accuracy : 0.9698
##
##        'Positive' Class : TP
##
```

```
# ROC Curve + AUC
pred_prob <- predict(final_rf, x_test, type = "prob")[,2]
roc_obj <- roc(y_test, pred_prob)

auc(roc_obj)
```

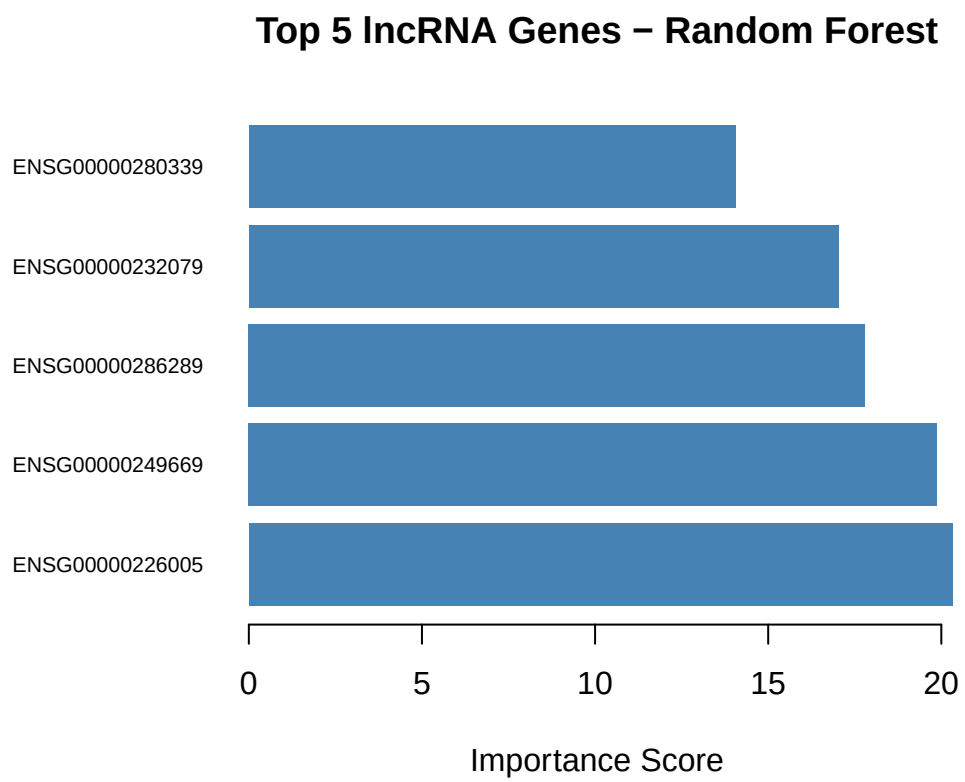
```
## Area under the curve: 0.9976
```

```
## Obtaining Biomarkers
# Extract Variable Importance
var_imp <- varImp(final_rf, scale = TRUE)
print(var_imp)
```

```
##                         TP        NT
## ENSG00000286289 17.795601 17.795601
## ENSG00000241684 11.548219 11.548219
## ENSG00000249669 19.870157 19.870157
## ENSG00000226005 20.313991 20.313991
## ENSG00000280339 14.055436 14.055436
## ENSG00000232079 17.038781 17.038781
## ENSG00000245812  9.216508  9.216508
## ENSG00000235904 13.598382 13.598382
## ENSG00000231246  9.830155  9.830155
## ENSG00000279204 12.438898 12.438898
```

```
var_imp_df <- as.data.frame(var_imp)
var_imp_df$Gene <- rownames(var_imp)

# Arrange by importance (ROC)
var_imp_df <- var_imp_df %>%
  arrange(desc(TP))

# Plot: VarImp plot for top 5 features
top5 <- var_imp_df %>%
  dplyr::slice(1:5)

par(mar = c(5, 12, 4, 2))

barplot(height = top5$TP,
        names.arg = top5$Gene,
        main = "Top 5 lncRNA Genes - Random Forest",
        xlab = "Importance Score",
        col = "steelblue",
        las = 1,
```

```
        horiz = TRUE,
        cex.names = 0.7,
        border = NA)
```




```
## Select the Top 5 features based on descending importance
rf_biomarkers <- top5

# Using a function to annotate genes - BioMart
annotate_genes <- function(biomarkers, model){

  # Remove NA and duplicates
  biomarkers <- biomarkers[!is.na(biomarkers)]
  biomarkers <- unique(biomarkers)

  # Remove empty strings
  biomarkers <- biomarkers[biomarkers != ""]

  # Biomart Connection
  mart <- useEnsembl(
  biomart = "genes",
  dataset = "hsapiens_gene_ensembl",
  mirror = "useast"
  )

  # Gene Annotations
```

```
  gene_annotation <- getBM(
    attributes = c("ensembl_gene_id", "hgnc_symbol"),
    filters = "ensembl_gene_id",
    values = biomarkers,
    mart = mart
  )

  # Create annotation dataframe
  ann_df <- data.frame(
    ensembl_gene_id = biomarkers,
    stringsAsFactors = FALSE
  ) %>%
    left_join(gene_annotation, by = "ensembl_gene_id")

  # Remove unannotated genes
  ann_df <- ann_df %>%
    filter(hgnc_symbol != "" & !is.na(hgnc_symbol))

  # Print biomarkers
  cat("Top", length(ann_df$hgnc_symbol), model, "Biomarkers:\n")
  print(ann_df$hgnc_symbol)
  cat("\n")

  return(ann_df)
}

rf_biomarkers_df <- annotate_genes(rf_biomarkers, "Random Forest")
```

```
## Top 3 Random Forest Biomarkers:
## [1] "LINC02660" "CARMN"     "LINC01697"
```

```
## Note: Only three biomarkers were annotated due to matches in BioMart only found for three ensembl ID
```

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 22.04.4 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.10.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.0  LAPACK version 3.10.0
##
## locale:
##  [1] LC_CTYPE=en_IN.UTF-8       LC_NUMERIC=C
##  [3] LC_TIME=en_IN.UTF-8        LC_COLLATE=en_IN.UTF-8
##  [5] LC_MONETARY=en_IN.UTF-8    LC_MESSAGES=en_IN.UTF-8
##  [7] LC_PAPER=en_IN.UTF-8       LC_NAME=C
##  [9] LC_ADDRESS=C               LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_IN.UTF-8 LC_IDENTIFICATION=C
##
## time zone: Asia/Kolkata
```

```
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] effsize_0.8.1        biomaRt_2.66.2       ROSE_0.0-4
## [4] pROC_1.19.0.1        randomForest_4.7-1.2 caret_7.0-1
## [7] lattice_0.22-5       ggplot2_4.0.2        dplyr_1.2.0
##
## loaded via a namespace (and not attached):
##  [1] DBI_1.3.0            httr2_1.2.2          rlang_1.1.7
##  [4] magrittr_2.0.4       otel_0.2.0           e1071_1.7-17
##  [7] compiler_4.5.2       RSQLite_2.4.6        png_0.1-9
## [10] vctrs_0.7.1          reshape2_1.4.5       stringr_1.6.0
## [13] pkgconfig_2.0.3      crayon_1.5.3         fastmap_1.2.0
## [16] dbplyr_2.5.2         XVector_0.50.0       rmarkdown_2.30
## [19] prodlim_2026.03.11   purrr_1.2.1          bit_4.6.0
## [22] xfun_0.56            cachem_1.1.0         progress_1.2.3
## [25] recipes_1.3.1        blob_1.3.0           parallel_4.5.2
## [28] prettyunits_1.2.0    R6_2.6.1             stringi_1.8.7
## [31] RColorBrewer_1.1-3   parallelly_1.46.1    rpart_4.1.24
## [34] lubridate_1.9.5      Rcpp_1.1.1           Seqinfo_1.0.0
## [37] iterators_1.0.14     knitr_1.51           future.apply_1.20.2
## [40] IRanges_2.44.0       Matrix_1.7-4         splines_4.5.2
## [43] nnet_7.3-20          timechange_0.4.0     tidyselect_1.2.1
## [46] rstudioapi_0.18.0    dichromat_2.0-0.1    yaml_2.3.12
## [49] timeDate_4052.112    codetools_0.2-19     curl_7.0.0
## [52] listenv_0.10.1       tibble_3.3.1         plyr_1.8.9
## [55] Biobase_2.70.0       withr_3.0.2          KEGGREST_1.50.0
## [58] S7_0.2.1             evaluate_1.0.5       future_1.70.0
## [61] survival_3.8-3       proxy_0.4-29         BiocFileCache_3.0.0
## [64] xml2_1.5.2           Biostrings_2.78.0    pillar_1.11.1
## [67] filelock_1.0.3       foreach_1.5.2        stats4_4.5.2
## [70] generics_0.1.4       hms_1.1.4            S4Vectors_0.48.0
## [73] scales_1.4.0         globals_0.19.1       class_7.3-23
## [76] glue_1.8.0           tools_4.5.2          data.table_1.18.2.1
## [79] ModelMetrics_1.2.2.2 gower_1.0.2          grid_4.5.2
## [82] ipred_0.9-15         AnnotationDbi_1.72.0 nlme_3.1-168
## [85] cli_3.6.5            rappdirs_0.3.4       lava_1.8.2
## [88] gtable_0.3.6         digest_0.6.39        BiocGenerics_0.56.0
## [91] farver_2.1.2         memoise_2.0.1        htmltools_0.5.9
## [94] lifecycle_1.0.5      hardhat_1.4.2        httr_1.4.8
## [97] bit64_4.6.0-1        MASS_7.3-65
```